SpecChain Part 4: Tail Contracts (I11–I13) Contraction, ISS/Tubes, and WSTS

Byte-replayable witnesses, termination under `EnvLock`, and acceptance obligations

Abstract

We strengthen Part 4 with a unifying TailContract interface and two meta-theorems that make *acceptance extensional in the tail bytes* and *safety-or-halt* immediate under the Part 3 No-Zeno discipline. Each family—**I11 Contraction**, **I12 ISS/Tubes**, and **I13 WSTS**—instantiates TailContract with (i) a canonical *witness-bytes* shape (`canon/v1` JSON), (ii) machine-checkable *obligations* and a progress measure coupled to the Loader's rank, and (iii) explicit *termination guarantees* compatible with the acceptance algebra **Q** and frontier accounting Δ_{fr}. We normalize *axis bindings*, *work bounds*, and *unknowns ledgering* across families and fix a uniform TailReceipt surface that prices work, binds evidence into the `proof_surface`, and preserves acceptance determinism.

1 Scope, Interfaces, and Acceptance Determinism

Assumption 1.1 (Replay discipline: `EnvLock`, `ReplayPlan`, canonical traversal)**.** All computations are executed under `EnvLock` and `ReplayPlan`: arithmetic, RNG, solver libraries, and serialization are pinned and byte-replayable under `canon/v1`. Each tail run exposes a *canonical witness sub-object* (the *tail-witness bytes*) inside the run's `proof_surface`. Families that explore a search/transition space must use a hash-stable, determinized traversal (e.g., BFS/DFS with lexicographic tie-breakers on canonical bytes); the chosen strategy is replay-pinned by `EnvLock`. Deterministic reject emits a `RejectionCert`; successful completion emits receipts and ledger updates.

Definition 1.2 (TailContract interface (family-independent))**.** A tail contract instance consists of the tuple

$$\mathsf{TailContract} \;=\; (\mathsf{Witness},\ \mathsf{Hyp},\ \mathsf{Obl},\ \mathsf{Exec},\ \mathsf{Step},\ \mathsf{Accept},\ \rho,\ \mathsf{Work})$$

with the following obligations, all machine-checkable from the tail-witness bytes embedded in `proof_surface`:

(a) **Witness & hypotheses.** Witness is canonical JSON (`canon/v1`) whose digest is bound into `proof_surface`. The hypothesis set Hyp (e.g., "$\gamma<1$", tube invariance premises, WQO/monotonicity certificates) is verified from Witness and the pinned environment.

(b) **Execution space.** Exec is a finite or effectively enumerable space of replay states; $\mathsf{Step} : \mathsf{Exec} \to \mathsf{Exec}$ (or a determinized exploration kernel) is total and `EnvLock`-pinned. If an internal work counter or traversal queue exists, its ordering is canonical.

(c) **Acceptance predicate.** $\mathsf{Accept} : \mathsf{Exec} \to \{\textsc{accept}, \textsc{continue}\}$ is total and must be *extensional in the tail-witness bytes*: two runs with identical tail-witness bytes and `EnvLock` identifiers yield the same Accept outcomes along the same determinized execution.

(d) **Progress measure.** $\rho : \mathsf{Exec} \to \mathbb{N}^k$ is a well-founded (lexicographic) rank that (i) decreases on every non-accepting Step that is permitted by the family's obligations, or (ii) when no such decrease occurs, consumes the Part 3 dwell budget (No-Zeno window) as a guard. Families expose ρ components so the Loader can couple them into the global rank.

(e) **Work model.** $\mathsf{Work} \in \{\texttt{iters_upper}, \texttt{steps_upper}, \texttt{nodes_upper}\}$ is a normalized expected-work bound used to price frontier debit Δ_{fr} against realized counters; its key and units are fixed by the family but the pricing kernel is shared.

(f) **Axis bindings.** The witness provides typed `axis_bindings` that map verified properties to deltas in the acceptance algebra **Q**; these adapters are monotone (stronger evidence ⇒ not-worse debit).

(g) **Unknowns discipline.** The instance declares `beta_unknowns_cap.bytes` and adheres to a guard-first rule: any priced unknown drives a deterministic failure path or is ledgered to a designated **Q** axis as the absorbing bottom; policy default is 0 bytes.

Remark 1.3 (Instantiation by families)**.** I11 (Contraction), I12 (ISS/Tubes), and I13 (WSTS) each instantiate TailContract by supplying concrete (Exec, Step, ρ, Work), hypotheses and checkers (e.g., Lipschitz/FP certificates; CLF/CBF/LMI receipts; WQO/monotonicity and acceleration proofs), and an Accept predicate. The instance's witness and receipt shapes are canonical and family-tagged.

Theorem 1.4 (Acceptance extensionality (tail bytes))**.** Under Assumption 1.1 and Definition 1.2, let two runs produce `proof_surface` objects that differ only outside the tail-witness sub-object and share the same `EnvLock` identifier. Then their acceptance outcomes coincide: the (determinized) execution traces are identical and Accept returns ACCEPT at the same step count (or never) in both.

Proof sketch. `EnvLock` pins all sources of nondeterminism, including traversal tie-breaking. All predicates read only `EnvLock` identity and the tail-witness bytes, which are identical by premise; thus the induced Step and Accept are extensionally equal. Hence the replayed trace and verdict coincide. □

Corollary 1.5 (Acceptance depends only on `proof_surface`)**.** Fix `EnvLock`. For any family $F \in \{\mathrm{I11}, \mathrm{I12}, \mathrm{I13}\}$, there exists a total function

$$\mathsf{Accept}_F : \{\texttt{proof_surface bytes}\} \to \{\textsc{accept}, \textsc{reject}\}$$

such that two runs with identical `proof_surface` yield identical verdicts. *(Immediate from Theorem 1.4 and the Loader's reject-on-guard semantics.)*

Theorem 1.6 (Safety-or-Halt (Tail meta-theorem))**.** Let a TailContract instance satisfy Definition 1.2 and run under the Part 3 No-Zeno discipline. Then along any replay, either

(i) Accept fires in finite steps (and a TailReceipt is emitted), or

(ii) a deterministic rejection occurs in finite steps (guard violation, hypothesis failure, or dwell exhaustion).

Proof sketch. By well-foundedness, any infinite sequence of non-accepting steps must either strictly decrease ρ infinitely often (impossible) or eventually stagnate. Under stagnation, the Loader's No-Zeno window consumes its finite dwell budget and forces a guard exit to rejection. Thus non-terminating, non-accepting executions are impossible. □

Trace bindings (cross-cutting). Each family's witness includes (i) `axis_bindings` that tie checks to **Q** axes and policy budgets, (ii) a normalized `expected_work_bound` in Work for pricing Δ_{fr}, and (iii) `beta_unknowns_cap` to ledger unknowns (-ledger). The emitted TailReceipt binds (a) the family ID and witness digest, (b) a verdict plus uniform failure code, (c) realized work counters priced into Δ_{fr}, (d) the unknowns byte-count against policy, and, for search-based families, (e) a compact exploration digest (e.g., visited-sequence hash). These bytes are included in `proof_surface`, preserving extensionality.

2 I11 Contraction Tail: Strong Convergence (FP-safe)

2.1 Model and Witness (effective FP contraction)

Definition 2.1 (Contraction setting with effective FP factor)**.** Let $(X, \|\cdot\|)$ be a normed space and $T : X \to X$ a map implemented under `EnvLock`. Fix a closed ball $\mathcal{B}_{\|\cdot\|}(x^\star, r) = \{x : \|x - x^\star\| \le r\}$. The implementation exposes an *effective FP contraction factor* $\tilde{\gamma} \in (0, 1)$ (upper-bounded in the witness by `gamma_fp`) such that, for all x, y in the ball, the `EnvLock`-replayed update satisfies

$$\|T(x) - T(y)\| \;\le\; \tilde{\gamma}\,\|x - y\|$$

as executed (i.e., rounding included). Forward invariance holds if $T(\mathcal{B}_{\|\cdot\|}(x^\star, r)) \subseteq \mathcal{B}_{\|\cdot\|}(x^\star, r)$ under `EnvLock`arithmetic.

Witness bytes (canonical, with certificates + trace bindings).

Listing 1: ContractionTail witness (FP-safe)

```
{
  "type": "ContractionTail",
  "norm_id": "l2",
  "center_x_star_digest": "blake3-256:...",
  "ball_radius": 0.125,

  "gamma_fp": 0.82,
  "lipschitz_cert_digest": "blake3-256:...",   // proves tilde{gamma} <=
     gamma_fp < 1 on the ball
  "invariance_cert_digest": "blake3-256:...",  // proves T(B) subseteq B
     under EnvLock
  "start_inside_cert_digest": "blake3-256:...",// proves ||x0 - x*|| <= r
     under EnvLock

  "residual_bound": 1e-6,
  "operator_commit": {
    "kind": "raw",
    "code_hash":   "blake3-256:...",
    "param_hash":  "blake3-256:..."
  },
  "init_state_digest": "blake3-256:...",
  "max_iters": 100000,

  "axis_bindings": [{ "axis_id": "Max(FixedPointError)", "budget_id": "
     B_FP" }],
  "expected_work_bound": { "iters_upper": 1500 },
  "beta_unknowns_cap": { "bytes": 0 }
}
```

Remark 2.2 (Operator specialization: gradient descent template)**.** When $T(x) = x - \eta\nabla f(x)$ for an L-smooth, μ-strongly convex f, the contract may use a structured commit:

```
"operator_commit": {
  "kind": "gradient_descent",
  "eta": 0.5,
  "mu": 0.2, "L": 1.6,
  "mu_proof_hash": "blake3-256:...",
```

```
  "L_proof_hash":  "blake3-256:..."
}
```

The checker derives a nominal $\gamma = \max\{|1-\eta\mu|, |1-\eta L|\} < 1$ and then verifies, via the `lipschitz_cert_digest`, that the *effective* $\tilde{\gamma} \le$ `gamma_fp` < 1 holds under `EnvLock`rounding.

Remark 2.3 (Variant I11′ (averaged maps))**.** Optionally, the witness may include "`averaged`$_{alpha}$" :$\alpha \in (0,1)$ and declare $x_{k+1} = (1-\alpha)x_k + \alpha T(x_k)$ with T nonexpansive (Krasnosel'skiĭ–Mann). Acceptance remains residual-based; rates are sublinear. The checker requires a corresponding certificate that the averaged operator is `EnvLock`-stable (details omitted here for brevity).

2.2 Obligations, Guarantees, and Work Pricing

I11 as a TailContract instance.

- $\mathsf{Exec} = \{(k, x_k) \mid k \in \mathbb{N}\}$ with determinized update $x_{k+1} = T(x_k)$ under `EnvLock`; $\mathsf{Step}(k, x_k) = (k{+}1, x_{k+1})$.
- $\mathsf{Accept}(k, x_k) =$ ACCEPT iff $r_k := \|x_k - x_{k-1}\| \le$ `residual_bound` for some $k \ge 1$.
- Progress measure $\rho(k, x_k) = (\lceil c \log(1 + r_k/\varepsilon) \rceil, \texttt{max_iters} - k)$ for a fixed $c > 0$ and $\varepsilon =$ `residual_bound`: while non-accepting, ρ decreases lexicographically (geometric decay then iteration guard).
- $\mathsf{Work} =$ `iters_upper`; realized iterations are priced to Δ_{fr} using the shared kernel.

Invariant 2.4 (Certificate checks (mechanized))**.** The checker must verify, from the witness bytes and `EnvLock`:

(i) *Effective contraction:* $\tilde{\gamma} \le$ `gamma_fp` < 1 on $\mathcal{B}_{\|\cdot\|}(x^\star, r)$ via `lipschitz_cert_digest`.

(ii) *Forward invariance:* $T(\mathcal{B}_{\|\cdot\|}(x^\star, r)) \subseteq \mathcal{B}_{\|\cdot\|}(x^\star, r)$ via `invariance_cert_digest`.

(iii) *Start inside:* $\|x_0 - x^\star\| \le r$ via `start_inside_cert_digest`.

(iv) *Deterministic stopping:* residual r_k is computed under `EnvLock`and compared to `residual_bound`.

Theorem 2.5 (I11 Safety-or-Halt)**.** If the certificate checks hold and $x_0 \in \mathcal{B}_{\|\cdot\|}(x^\star, r)$, then the I11 instance satisfies Theorem 1.6: along any replay, either acceptance fires in finite iterations or a deterministic rejection is produced (certificate failure or `max_iters` guard).

Lemma 2.6 (Explicit residual bound)**.** Let $d_0 = \|x_0 - x^\star\|$ and suppose Definition 2.1 with $\tilde{\gamma} \le$ `gamma_fp` < 1. Then the `EnvLock`-residuals satisfy

$$r_k = \|x_k - x_{k-1}\| \;\le\; (1+\tilde{\gamma})\tilde{\gamma}^{\,k-1} d_0 \;\le\; (1+\texttt{gamma_fp})\texttt{gamma_fp}^{\,k-1} d_0.$$

A sufficient iteration bound for acceptance ($r_k \le \varepsilon :=$ `residual_bound`) is

$$k \;\ge\; 1 + \left\lceil \frac{\log\!\left(\frac{\varepsilon}{(1+\texttt{gamma_fp})d_0}\right)}{\log(\texttt{gamma_fp})} \right\rceil,$$

interpreting $\log(\texttt{gamma_fp}) < 0$. If d_0 is unknown but bounded by r, use $d_0 \le r$.

Receipt (family-specific fields). Upon acceptance or deterministic reject, the TailReceipt records: (i) family ID `"I11_Contraction"`, (ii) witness digest, (iii) verdict and failure code (if any), (iv) realized iteration count and priced Δ_{fr} w.r.t. `iters_upper`, (v) the verified upper bound `gamma_fp` and the certificate digests used, and (vi) the β-unknowns byte-count (must not exceed the cap).

2.3 Failure Codes (uniformized)

- **C1_FP_CONTRACTION_FAIL**: missing/invalid Lipschitz certificate or `gamma_fp` ≥ 1 on the committed ball.
- **C2_START_OUTSIDE**: start-inside certificate fails ($\|x_0 - x^\star\| > r$).
- **C3_NOT_FORWARD_INVARIANT**: invariance certificate fails ($T(B) \not\subseteq B$).
- **C4_MAX_ITERS**: exceeded `max_iters` without meeting `residual_bound`.
- **C5_COMMIT_MISMATCH**: operator/param/init digests mismatch `operator_commit/init_state_digest`.
- **C6_UNKNOWN_BUDGET**: `beta_unknowns_cap.bytes` exceeded (guard-first failure).

2.4 Checklist (I11 obligations under TailContract)

C1. Validate schema and canonicalization of the witness (`canon/v1`); bind digest into `proof_surface`.

C2. Verify `lipschitz_cert_digest` $\Rightarrow \tilde{\gamma} \leq$ `gamma_fp` < 1 on $\mathcal{B}_{\|\cdot\|}(x^\star, r)$.

C3. Verify `invariance_cert_digest` $\Rightarrow$ forward invariance of the ball under `EnvLock`.

C4. Verify `start_inside_cert_digest` and compute d_0 (or $d_0 \leq r$) under `EnvLock`.

C5. Run determinized updates; accept on $r_k \leq$ `residual_bound`; otherwise enforce monotone counter and `max_iters`.

C6. Price work: compare realized iterations to `iters_upper` and debit Δ_{fr}; bind axis deltas from `axis_bindings`.

C7. Ledger unknowns (must not exceed cap); emit uniform failure code on violation.

3 I12 ISS/Tubes Tail: Lyapunov & Invariant Tubes (robust & probabilistic)

3.1 Model and Witness (Q'/uncertainty aware)

Definition 3.1 (ISS with disturbances and observation error)**.** Consider the `EnvLock`-executed system

$$x_{k+1} = f(x_k, \pi(x_k), d_k), \qquad y_k = x_k + e_k,$$

with control policy π, disturbance d_k and observation error e_k. We say the closed-loop is *ISS* if $\exists\ \beta \in \mathcal{KL},\ \gamma_d, \gamma_e \in \mathcal{K}_\infty$ such that for all k:

$$\|x_k\| \ \leq\ \beta(\|x_0\|, k) \ +\ \gamma_d\Big(\sup_{0\leq j<k} \|d_j\|\Big) \ +\ \gamma_e\Big(\sup_{0\leq j<k} \|e_j\|\Big).$$

A *tube* $\mathcal{T} \subseteq X$ is forward invariant under π (robustly in $\mathbf{Q}'$) if, for all admissible (d_k, e_k) in the declared $\mathbf{Q}'$ carrier, $x_k \in \mathcal{T} \Rightarrow x_{k+1} \in \mathcal{T}$.

Witness bytes (canonical; robust & probabilistic options).

Listing 2: ISSTail witness (Q'/uncertainty aware)

```
{
  "type": "ISSTail",

  "clf_cbf_structure": {
    "V_code_hash": "blake3-256:...",
    "alpha_lower": 0.01, "alpha_upper": 1.0,   // CLF bounds
    "rho_decay": 0.12,                          // V(x_{k+1})-V(x_k) <= -
       rho V(x_k) + ...
    "sigma_forms": ["affine_disturbance","affine_obs"] // form of gains in
        certificate
  },

  "uncertainty_model": {
    "disturbance": { "kind": "deterministic", "norm": "l2", "max": 0.05 },
    "observation": { "norm": "l2", "max": 0.01 },
    "Qprime_stance": { "carrier": "intervals", "semantics": "robust" }
  },
  // --- Optional probabilistic variant (I12-SG) replaces the block above:
  // "uncertainty_model": {
  //   "disturbance": { "kind": "subgaussian", "psi2": 0.04 },  //
     variance proxy
  //   "observation": { "kind": "bounded", "norm": "l2", "max": 0.01 },
  //   "Qprime_stance": { "carrier": "subgaussian", "semantics": "high-
     prob", "delta_max": 1e-6 }
  // },

  "tube": { "shape": "polytope", "A_hash": "blake3-256:...", "b_hash": "
     blake3-256:..." },

  "policy_commit": { "code_hash": "blake3-256:..." },

  "dwell_min_steps": 3,
  "accept_criterion": { "in_tube_steps": 20 },

  // Linear/LMI (optional accelerator for verification)
  "lmi_witness": {
    "enabled": true,
    "problem_digest": "blake3-256:...",      // canonical bytes for the
       LMI
    "solver_commit": { "code_hash": "blake3-256:..." },
    "rounding_receipt_hash": "blake3-256:..."// proves feasibility
       survives FP rounding
  },

  "axis_bindings": [
    { "axis_id": "Max(ConstraintViolation)", "budget_id": "B_CONSTR" },
    { "axis_id": "P(UnsafeExit)",            "budget_id": "B_EXIT"   }
  ],
```

```
  "expected_work_bound": { "steps_upper": 10000 },
  "beta_unknowns_cap": { "bytes": 0 }
}
```

Remark 3.2 (Deterministic vs. probabilistic stance)**.** The `uncertainty_model` declares the $\mathbf{Q}'$ carrier and semantics. For `"deterministic"/"intervals"`, tube invariance must hold for *all* admissible (d_k, e_k). For `"subgaussian"`, the checker derives a high-probability tube-invariance bound with failure probability $\leq$ `Qprime_stance.delta_max`; this debit is recorded on the `P(UnsafeExit)` axis.

3.2 Obligations, Guarantees, and Work Pricing

I12 as a TailContract instance.

- $\mathsf{Exec} = \{(k, x_k, s)\}$ where s is the local dwell counter; Step applies the `EnvLock`-pinned π and f, updates x_{k+1} and s.
- $\mathsf{Accept}(k, x_k, s) = \textsc{accept}$ iff $x_k \in \mathcal{T}$ for the last $K =$ `accept_criterion.in_tube_steps` steps, the local dwell constraint is satisfied ($s \geq$ `dwell_min_steps`), and (probabilistic stance only) the certified exit probability $\widehat{p}_{\text{exit}} \leq$ `Qprime_stance.delta_max`.
- Progress measure $\rho(k, x_k, s) = (\underbrace{\max\{0, K - \text{runlen}_{\mathcal{T}}(k)\}}_{\text{in-tube backlog}}, \underbrace{\max\{0, \texttt{dwell_min_steps} - s\}}_{\text{dwell backlog}})$, lex-ordered, coupled to the Loader's dwell budget (No-Zeno).
- $\mathsf{Work} =$ `steps_upper`; realized steps priced to Δ_{fr} by the shared kernel.

Invariant 3.3 (CLF/CBF certificate & tube invariance in $\mathbf{Q}'$)**.** The checker verifies, from witness bytes, that there exists a function V (code-hash committed) s.t.

(i) **Bounds:** $\alpha_{\min}\|x\|^2 \leq V(x) \leq \alpha_{\max}\|x\|^2$ with $\alpha_{\min} =$ `alpha_lower` and $\alpha_{\max} =$ `alpha_upper`.

(ii) **Decay:** for all admissible (d_k, e_k) in the declared $\mathbf{Q}'$ carrier,

$$V(x_{k+1}) - V(x_k) \;\leq\; -\rho V(x_k) \;+\; \sigma_d(\|d_k\|) \;+\; \sigma_e(\|e_k\|),$$

with $\rho =$ `rho_decay` > 0 and gain forms committed by `sigma_forms`.

(iii) **Tube invariance:** if $x_k \in \mathcal{T}$, then $x_{k+1} \in \mathcal{T}$ robustly (*deterministic* stance) or with certified failure probability $\leq$ `Qprime_stance.delta_max` (*subgaussian* stance).

(iv) **Dwell compliance:** the local dwell counter s enforces a minimum hold; when s reaches `dwell_min_steps`, a *DwellReceipt* component is prepared that authorizes the Loader to reset its global dwell window.

Theorem 3.4 (I12 Safety-or-Halt)**.** If Invariant 3.3 holds and the witness hashes match the execution (`policy_commit`, tube matrices, etc.), then the I12 instance satisfies Theorem 1.6: along any replay, either (i) Accept fires after at most $K +$ `dwell_min_steps` additional in-tube steps, or (ii) a deterministic rejection occurs (certificate failure, tube exit, dwell violation, risk above threshold, or guard exhaustion).

Corollary 3.5 (Linear case with LMI receipt)**.** For $x_{k+1} = Ax_k + Bd_k$ with $\|d_k\| \leq \bar{d}$ and polytope tube $\mathcal{T} = \{x : A_{\mathcal{T}} x \leq b\}$, take $V(x) = x^\top P x$, $P \succ 0$. Robust decay reduces to LMIs; tube invariance reduces to linear containments. If `lmi_witness.enabled=true`, the checker must:

(a) validate `problem_digest` and `solver_commit` under `EnvLock`,

(b) verify the `rounding_receipt_hash` proving feasibility survives rounding (eigenvalue lower bounds < 0),

(c) bind P's digest and minimal slack margins into the TailReceipt.

Probabilistic stance (I12-SG) details. Under `disturbance.kind="subgaussian"`, the checker computes a reachable-set tail bound (via the committed gain forms) that yields a certified exit probability $\widehat{p}_{\text{exit}}$. Acceptance requires both K in-tube steps and $\widehat{p}_{\text{exit}} \le$ `Qprime_stance.delta_max`. The debit $\text{debit}_{\mathbf{Q}}[\texttt{P(UnsafeExit)}] \leftarrow \widehat{p}_{\text{exit}}$ is recorded in the TailReceipt.

Receipt (family-specific fields). On acceptance or deterministic reject, the TailReceipt records: (i) family `"I12_ISS_Tubes"`, (ii) witness digest, (iii) verdict and uniform failure code, (iv) realized step count priced to Δ_{fr} vs. `steps_upper`, (v) dwell summary (local counts and DwellReceipt authorization), (vi) invariance margins (deterministic) or $\widehat{p}_{\text{exit}}$ (probabilistic), (vii) LMI/solver digests if used, and (viii) β-unknowns byte-count.

3.3 Failure Codes (uniformized)

- **T1_CLF_BOUNDS**: V violates $\alpha_{\min}\|x\|^2 \le V(x) \le \alpha_{\max}\|x\|^2$.
- **T2_DECAY_FAIL**: decay inequality fails for some admissible (d_k, e_k) in the declared $\mathbf{Q}'$ carrier.
- **T3_TUBE_NOT_INVARIANT**: tube invariance fails (robust) or certified risk bound not provable.
- **T4_DWELL_VIOLATION**: local dwell requirement not met when required to accept; or misuse of dwell reset.
- **T5_COMMIT_MISMATCH**: policy/tube/certificate hash mismatch against witness.
- **T6_LMI_RECEIPT_INVALID**: inconsistent `lmi_witness` (problem or rounding receipt invalid).
- **T7_RISK_ABOVE_THRESH**: $\widehat{p}_{\text{exit}} >$ `Qprime_stance.delta_max` (prob. stance).
- **T8_UNKNOWN_BUDGET**: `beta_unknowns_cap.bytes` exceeded (guard-first failure).

3.4 Checklist (I12 obligations under TailContract)

T1. Validate schema/canonicalization (`canon/v1`) and bind witness digest into `proof_surface`.

T2. Verify CLF bounds and decay with declared σ forms and $\mathbf{Q}'$ carrier; compute certified gains.

T3. Prove tube forward invariance in the declared stance (robust or sub-Gaussian high-prob); compute margins or $\widehat{p}_{\text{exit}}$.

T4. Enforce dwell: maintain local counter s; generate DwellReceipt when threshold reached; couple to Loader No-Zeno.

T5. Run determinized updates; accept only after K consecutive in-tube steps (and risk $\le$ threshold if probabilistic).

T6. If `lmi_witness.enabled`: verify `problem_digest`, `solver_commit`, and `rounding_receipt_hash`; bind P digest and slacks.

T7. Price realized steps against `steps_upper` into Δ_{fr}; apply `axis_bindings` as typed adapters into $\mathbf{Q}$.

T8. Ledger unknowns bytes; reject with **T8_UNKNOWN_BUDGET** on overrun.

4 I13 WSTS Tail: Well-Structured Termination (deterministic exploration)

4.1 Model and Witness (order/encoder/acceleration certified)

Definition 4.1 (Well-Structured Transition System (WSTS) with certified encoding)**.** A WSTS is $(S, \to, \preceq)$ where $\preceq$ is a well-quasi-order (wqo) and $\to$ is monotone: $s_1 \preceq s_2$ and $s_1 \to t_1$ imply $\exists t_2$ with $s_2 \to t_2$ and $t_1 \preceq t_2$. A *state encoder* $E : \text{Concrete} \to S$ is *monotone* if $x \preceq_{\text{conc}} y \Rightarrow E(x) \preceq E(y)$; the tail commits to E and proves its monotonicity.

Witness bytes (canonical; deterministic traversal + sound acceleration).

Listing 3: WSTSTail witness (deterministic + certified)

```
{
  "type": "WSTSTail",

  "state_commit": {
    "encoder_hash": "blake3-256:..."
  },

  "order_proof": {
    "wqo_certificate_hash": "blake3-256:..."
  },

  "monotonicity_proof": {
    "transition_checker_hash": "blake3-256:...",    // checks system
       monotonicity wrt. the wqo
    "encoder_monotonicity_hash": "blake3-256:..."   // proves E is
       monotone wrt. the wqo
  },

  "exploration_strategy": {
    "kind": "bfs",                // {"bfs" | "dfs"}
    "tie_break": "lex_bytes"      // deterministic tie-break on canonical
       bytes
  },

  "acceleration": {
    "kind": "finite-basis",       // {"finite-basis" | "widening" | "other
       "}
    "soundness_cert_hash": "blake3-256:..." // proves accelerated step is
       an upward-closed over-approx
  },

  "initial_state_digest": "blake3-256:...",
```

```
  "cover_goal": {
    "representation": "antichain_minimals",
    "minimals_hash": "blake3-256:...",   // canonical bytes of the list of
        minimal basis elements
    "minimals_count": 3
  },

  "max_explored": 100000,

  "axis_bindings": [
    { "axis_id": "Max(UnsafeReachable)", "budget_id": "B_REACH" }
  ],
  "expected_work_bound": { "nodes_upper": 100000 },
  "beta_unknowns_cap": { "bytes": 0 }
}
```

Remark 4.2 (Upward-closed goals via minimal antichains)**.** An upward-closed set $\mathcal{U} \subseteq S$ is uniquely determined by the antichain of its $\preceq$-minimal elements $\min(\mathcal{U})$. The witness commits to the canonical bytes of this minimal basis (`cover_goal.minimals_hash`); a state s is in $\mathcal{U}$ iff $\exists u \in \min(\mathcal{U})$ with $u \preceq s$.

4.2 Obligations, Termination, and Work Pricing

I13 as a TailContract instance.

- $\mathsf{Exec} = \{(\text{Frontier}, \text{Visited}, \text{Basis}, n)\}$ with $n \in \mathbb{N}$ the node counter. Step deterministically pops from `Frontier` using `exploration_strategy`, applies $\rightarrow$, accelerates (if enabled) to update Basis (finite-basis or widening), prunes by upward closure, and pushes children in canonical order; all byte-replayable under `EnvLock`.
- Accept fires iff the upward closure of Basis intersects the goal, i.e., $\uparrow\text{Basis} \cap \mathcal{U} \neq \varnothing$, where $\mathcal{U}$ is given by the committed goal antichain.
- Progress measure $\rho = (\underbrace{g(\text{Frontier}, \text{Basis})}_{\text{frontier backlog}}, \underbrace{\texttt{max_explored} - n}_{\text{node budget}})$, lex-ordered, where g is any canonical, well-founded measure of pending work (e.g., frontier level + antichain size). Non-accepting steps strictly decrease ρ until either Accept or budget exhaustion.
- Work = `nodes_upper`; realized node count priced to Δ_{fr} by the shared kernel.

Invariant 4.3 (Certificates and deterministic traversal)**.** From the witness bytes, the checker must verify:

(i) **WQO validity:** `order_proof.wqo_certificate_hash` certifies $\preceq$ is a wqo on S.

(ii) **Monotonicity:** transition monotonicity wrt. $\preceq$, and encoder monotonicity of E (`monotonicity_proof` hashes).

(iii) **Acceleration soundness:** the chosen `acceleration.kind` is sound; the certificate proves that each accelerated successor over-approximates a (possibly infinite) set of concrete successors as an upward-closed set.

(iv) **Canonical traversal:** the exploration strategy and tie-break are as declared; the queue/set bytes are ordered by `lex_bytes`.

(v) **Goal schema:** the goal's minimal antichain is well-formed (no comparable pair; canonicalized), consistent with the wqo.

Theorem 4.4 (I13 Safety-or-Halt)**.** If Invariant 4.3 holds, then the I13 instance satisfies Theorem 1.6: along any replay, either (i) the algorithm discovers a finite basis whose upward closure intersects the goal and Accept fires, or (ii) a deterministic rejection occurs (certificate failure, `max_explored` exhaustion, or other guard).

Remark 4.5 (Exploration fingerprint)**.** The emitted TailReceipt includes a *visited-sequence hash* computed over the canonical digests of visited/expanded items in traversal order. This fingerprint pins the entire search and ensures acceptance extensionality.

Receipt (family-specific fields). On acceptance or deterministic reject, the TailReceipt records: (i) family `"I13_WSTS"`, (ii) witness digest, (iii) verdict and uniform failure code, (iv) nodes explored and priced Δ_{fr} vs. `nodes_upper`, (v) `exploration_strategy` echo, visited-sequence hash, and final basis digest/size, (vi) acceleration soundness certificate hash, and (vii) β-unknowns byte-count.

4.3 Failure Codes (uniformized)

- **W1_WQO_INVALID**: invalid wqo certificate or mismatch with the committed order.
- **W2_MONOTONICITY_FAIL**: transition or encoder monotonicity proof fails.
- **W3_ACCEL_UNSOUND**: acceleration soundness certificate invalid or absent for the declared kind.
- **W4_GOAL_SCHEMA_INVALID**: goal minimal antichain ill-formed (non-canonical or comparable elements).
- **W5_EXPLORATION_LIMIT**: exceeded `max_explored` without acceptance.
- **W6_COMMIT_MISMATCH**: encoder/order/goal bytes or strategy/tie-break mismatch against witness.
- **W7_UNKNOWN_BUDGET**: `beta_unknowns_cap.bytes` exceeded (guard-first failure).

4.4 Checklist (I13 obligations under TailContract)

W1. Validate witness schema and canonicalization (`canon/v1`); bind digest into `proof_surface`.

W2. Check wqo certificate; verify transition and encoder monotonicity checkers.

W3. Verify acceleration soundness; reject if certificate is missing or mismatched to `kind`.

W4. Validate goal antichain (no comparabilities; canonical minimal set consistent with $\preceq$).

W5. Run the determinized search with committed `exploration_strategy` and tie-break; maintain node counter n.

W6. Accept when $\uparrow$Basis intersects the goal; otherwise continue until `max_explored` then raise **W5_EXPLORATION_LIMIT**.

W7. Price realized nodes against `nodes_upper` into Δ_{fr}; apply `axis_bindings` into $\mathbf{Q}$.

W8. Compute and bind visited-sequence hash and final basis digest; ledger unknowns and enforce cap.

5 Uniform TailReceipt, Pricing Kernel, and "Toy Kit"

5.1 Normative TailReceipt (family-agnostic)

Definition 5.1 (TailReceipt (canonical, family-tagged))**.** Every TailContract instance (I11/I12/I13) emits a canonical JSON object TailReceipt whose digest is included in the run's `proof_surface`. The required fields are:

- `family_id` $\in$ {`"I11_Contraction"`, `"I12_ISS_Tubes"`, `"I13_WSTS"`}.
- `witness_digest`: BLAKE3-256 of the canonical witness bytes.
- `envlock_id`: identifier of the `EnvLock`/`ReplayPlan`context.
- `verdict`: `"ACCEPT"` or `"REJECT"`, with `failure_code` on reject (uniform codes per family).
- `work_counters`: realized counters with a single active key among `iters_realized` (I11), `steps_realized` (I12), `nodes_explored` (I13).
- `debit_receipt`: pricing of realized work against `expected_work_bound` into Δ_{fr} (Sec. 5.2); includes policy and tariff digests.
- `axis_debits`: map from `axis_bindings` to per-axis deltas in $\mathbf{Q}$ (typed adapters).
- `unknowns_bytes`: realized β-unknowns ledger (must satisfy the cap in the witness).
- `certs_digests`: multiset of certificate digests consumed (e.g., Lipschitz/invariance/start-inside; LMI/rounding; WQO/monotonicity/acceleration).
- `exploration_profile` (optional): family-specific trace digests, e.g., `visited_sequence_hash` (I13) or `dwell_receipt` (I12).

All fields are canonicalized under `canon/v1`; TailReceipt bytes are bound into `proof_surface`, preserving Theorem 1.4.

Listing 4: TailReceipt (illustrative shape)

```
{
  "family_id": "I12_ISS_Tubes",
  "witness_digest": "blake3-256:...",
  "envlock_id": "envlock:canon/v1:...",
  "verdict": "ACCEPT",
  "failure_code": null,

  "work_counters": { "steps_realized": 8731 },
  "debit_receipt": {
    "tariff_digest": "blake3-256:...",
    "expected_work_bound": { "steps_upper": 10000 },
    "priced_delta_fr": 0.87
  },
```

```
  "axis_debits": {
    "Max(ConstraintViolation)": 0,
    "P(UnsafeExit)": 9.4e-7
  },

  "unknowns_bytes": 0,
  "certs_digests": [
    "blake3-256:...V_code...", "blake3-256:...tube...", "blake3-256:...lmi
       ...",
    "blake3-256:...rounding..."
  ],

  "exploration_profile": {
    "dwell_receipt": { "local_dwell": 3, "authorized_reset": true }
  }
}
```

5.2 Pricing kernel and adapters into Q

Definition 5.2 (Normalized work model & kernel)**.** Let $\mathsf{WorkBound} \in \{\texttt{iters_upper}, \texttt{steps_upper}, \texttt{nodes_upper}\}$ be declared in the witness, and let $w_{\max}$ be the bound's value with realized work $w_{\text{real}} \in \mathbb{N}$. The *pricing kernel* is a monotone function $\mathsf{Price} : \mathbb{N} \times \mathbb{N} \to \mathbb{R}_{\geq 0}$ that computes a frontier debit $\mathsf{Price}(w_{\text{real}}, w_{\max}) = \Delta_{\text{fr}}$, subject to:

(a) **Monotonicity:** $w_{\text{real}} \leq w'_{\text{real}} \Rightarrow \mathsf{Price}(w_{\text{real}}, w_{\max}) \leq \mathsf{Price}(w'_{\text{real}}, w_{\max})$.

(b) **Budget domination:** if $w_{\text{real}} \leq w_{\max}$ then $\mathsf{Price}(w_{\text{real}}, w_{\max}) \leq \mathsf{Price}(w_{\max}, w_{\max})$.

(c) **Replayability:** the tariff and all inputs to Price are `EnvLock`-pinned bytes.

Lemma 5.3 (Typed adapters into **Q** are monotone)**.** Let ϕ be a declared adapter from verified properties to an axis $A \in \mathsf{Axis}$ of **Q**. If the witness W_1 is stronger than W_2 for that property (per the family's order), then $\phi(W_1) \preceq_{\mathbf{Q}} \phi(W_2)$. Hence `axis_debits` in TailReceipt are monotone in witness strength.

Remark 5.4 (Dwell coupling (I12) and exploration fingerprints (I13))**.** For I12, the `dwell_receipt` authorizes the Loader to reset its No-Zeno window after the local dwell threshold is met; the receipt records the local count and a boolean authorization bit. For I13, the optional `visited_sequence_hash` pins the search order and frontier/backlog history as a compact fingerprint.

5.3 "Toy kit": minimal witnesses → expected receipts

Contraction (I11, FP-safe).

- *Witness sketch:* $T(x) = 0.5x + 0.1$ on $\mathcal{B}_{\|\cdot\|}(0, 1)$ with certificates: `gamma_fp`$= 0.5$, valid Lipschitz & invariance digests, residual 10^{-6}, `iters_upper`$= 1500$.
- *Acceptance:* by Lemma 2.6 with $d_0 \leq 1$, $k \leq 1 + \lceil \log(10^{-6}/(1+0.5))/\log(0.5) \rceil = 1 + \lceil \log(6.67\times10^{-7})/\log(0.5) \rceil$.
- *Receipt:* `verdict="ACCEPT"`, `iters_realized`$= k$, priced $\Delta_{\text{fr}} \leq \mathsf{Price}(k, 1500)$, cert digests include Lipschitz/invariance/start-inside; `unknowns_bytes`$= 0$.

ISS/Tubes (I12, deterministic stance).

- *Witness sketch:* $x_{k+1} = Ax_k + Bd_k$, $\|d_k\| \le 0.05$, polytope tube, quadratic V with LMI witness and rounding receipt; `in_tube_steps` $= 20$, `dwell_min_steps` $= 3$, `steps_upper` $= 10^4$.
- *Acceptance:* after K+`dwell_min_steps` $= 23$ consecutive in-tube steps (Theorem 3.4).
- *Receipt:* `verdict="ACCEPT"`, `steps_realized`≥ 23, $\Delta_{\text{fr}} = \mathsf{Price}(\cdot, 10^4)$, `axis_debits` shows 0 on `Max(ConstraintViolation)`, LMI and rounding digests recorded, `dwell_receipt` authorizes No-Zeno reset.

ISS/Tubes (I12-SG, probabilistic stance).

- *Witness sketch:* sub-Gaussian disturbance with ψ_2=0.04, `Qprime_stance.delta_max` $= 10^{-6}$, same tube; $K = 20$, dwell $= 3$.
- *Acceptance:* if certified $\widehat{p}_{\text{exit}} \le 10^{-6}$ and 23 consecutive in-tube steps observed.
- *Receipt:* `P(UnsafeExit)` debit reported as $\widehat{p}_{\text{exit}}$ (typically $\le 10^{-6}$).

WSTS (I13, finite-basis).

- *Witness sketch:* VAS on $\mathbb{N}^n$ with Dickson wqo certificate; BFS/`lex_bytes` traversal; finite-basis acceleration with soundness proof; upward-closed goal as minimal antichain; `nodes_upper`$= 10^5$.
- *Acceptance:* when the discovered finite basis' upward closure intersects the goal (Theorem 4.4).
- *Receipt:* `verdict="ACCEPT"`, nodes explored $n \le 10^5$, priced Δ_{fr}, final basis digest and `visited_sequence_hash` present; `axis_debits` on `Max(UnsafeReachable)` reflects coverage decision.

Remark 5.5 (Policy defaults)**.** Unless overridden by policy, `beta_unknowns_cap.bytes`$= 0$ for all families; any encounter with priced unknowns causes **C6/T8/W7_UNKNOWN_BUDGET** rejection. Tariffs and adapter homomorphisms into **Q** are versioned artifacts whose digests are echoed in `debit_receipt` to ensure replayable pricing.

6 Comparison Table: Hypotheses vs. Guarantees (with Progress & Complexity)

7 Proof Obligations (TailContract + Per-Family Checklists)

Goal. This section fixes a *single, family-agnostic* checklist for any TailContract instance and then points to the per-family checklists already stated in Sections 2–4. The Loader enforces that all **TC**-items pass before accepting any family verdict as final.

Unified TailContract obligations (family-agnostic)

TC1. Schema & canonization. Validate the witness against its JSON Schema; canonicalize under `canon/v1`; bind `witness_digest` into `proof_surface`. Echo `envlock_id`.

TC2. Commit fidelity. Verify all code/param commits referenced by the witness (operators, policies, encoders, solvers). Any digest mismatch triggers uniform `_COMMIT_MISMATCH`.

Family	Hypotheses (machine-checked from witness)	Guarantees under `EnvLock` & TailContract	Work/Complexity
I11 Contraction (FP-safe)	• `gamma_fp` < 1 with `lipschitz_cert_digest` proving $\tilde{\gamma} \le$ `gamma_fp` on the committed ball • Forward invariance cert $T(B) \subseteq B$; start-inside cert $\|x_0 - x^\star\| \le r$ • `operator_commit` hashes; residual rule; schema/canon	• Geometric residual decay; accept when $r_k \le \varepsilon$ (Lemma 2.6) • Safety-or-Halt (Thm. 1.6); extensional verdict (Thm. 1.4) • Fail-closed on cert/guard violations; uniform receipt with Δ_{fr} pricing	• $O(\log(1/\varepsilon))$ iterations • Work key: `iters_upper`
I12 ISS/Tubes (robust / probabilistic)	• CLF bounds & decay (`alpha_lower/upper`, `rho_decay`, σ-forms) • Tube invariance in $\mathbf{Q}'$ per `uncertainty_model` (robust or sub-Gaussian, $\delta_{\max}$) • `policy_commit`; dwell discipline; optional LMI witness + rounding receipt	• Robust stance: $x_k \in \mathcal{T} \Rightarrow x_{k+1} \in \mathcal{T}$ for all admissible (d, e) • Prob. stance: accept only if $\widehat{p}_{\text{exit}} \le \delta_{\max}$ (debited on `P(UnsafeExit)`) • Accept after K in-tube steps and dwell met (Thm. 3.4); Safety-or-Halt applies	• $O(K +$ `dwell_min_steps` checks (plus cert verification) • Work key: `steps_upper`
I13 WSTS (deterministic exploration)	• WQO certificate; transition monotonicity checker; *encoder* monotonicity certificate • `exploration_strategy` (BFS/DFS, `lex_bytes` tie-break) committed • Acceleration soundness (finite-basis/widening); goal as minimal antichain (canonical)	• Deterministic coverability: accept iff $\uparrow$ Basis $\cap\, \mathcal{U} \neq \varnothing$ (Thm. 4.4) • Exploration fingerprint (visited-sequence hash) ensures extensionality (Thm. 1.4) • Safety-or-Halt with guard on `max_explored`	• Bounded by `max_explored` nodes • Work key: `nodes_upper`

Table 1: Tail families as TailContract instances: hypotheses, guarantees, and indicative work/complexity. All pricing uses the normalized kernel of Sec. 5.2; receipts are canonical and bound into `proof_surface`.

TC3. Determinized execution. Check that the declared Exec and Step (and any traversal queues) are `EnvLock`-pinned and hash-stable (e.g., BFS/DFS with `lex_bytes` tie-break for search).

TC4. Acceptance extensionality. Ensure the instance's Accept is a pure function of tail-witness bytes and `EnvLock` identity (Theorem 1.4); no hidden clocks or non-canonical sources.

TC5. Progress & No-Zeno coupling. Expose a well-founded ρ that decreases on non-accepting steps or, upon stagnation, consumes the Loader's dwell budget; reject deterministically on guard exhaustion (Theorem 1.6).

TC6. Work pricing kernel. Normalize `expected_work_bound` to one active key among `iters_upper`/`steps_upper`/`nodes_upper`; compute realized counters; price Δ_{fr} via the shared kernel (Def. 5.2). Bind tariff/parameter digests in the TailReceipt.

TC7. Axis adapters into Q. Apply typed, monotone adapters from verified properties to `axis_debits` (Lemma 5.3); include adapter/parameter digests in the TailReceipt.

TC8. Unknowns discipline. Enforce guard-first handling of priced unknowns; ledger `unknowns_bytes` and reject on cap breach. Default policy requires `beta_unknowns_cap.bytes`=0.

TC9. Uniform failure codes. Map any violation to the family's code set (I11: C1–C6; I12: T1–T8; I13: W1–W7) and include `failure_code` in the TailReceipt on reject.

TC10. Receipt binding. Emit a canonical TailReceipt (Def. 5.1); bind its bytes (including cert digests and exploration fingerprints, if any) into `proof_surface`.

Per-family obligations (pointers to checklists)

I11 Contraction (FP-safe):
Checklist **C1–C7** in Subsec. 2.4. Key items: effective FP contraction certificate (`lipschitz_cert_digest`); forward-invariance and start-inside certificates; deterministic residual rule; pricing by `iters_upper`. Failure codes: **C1_FP_CONTRACTION_FAIL**, **C2_START_OUTSIDE**, **C3_NOT_FORWARD_INVARIANT**, **C4_MAX_ITERS**, **C5_COMMIT_MISMATCH**, **C6_UNKNOWN_BUDGET**.

I12 ISS/Tubes (robust/prob.):
Checklist **T1–T8** in Subsec. 3.4. Key items: CLF bounds/decay in declared $\mathbf{Q}'$ carrier; tube invariance (robust or high-prob); dwell discipline and DwellReceipt; optional LMI problem + rounding receipt; pricing by `steps_upper`. Failure codes: **T1_CLF_BOUNDS**, **T2_DECAY_FAIL**, **T3_TUBE_NOT_INVARIANT**, **T4_DWELL_VIOLATION**, **T5_COMMIT_MISMATCH**, **T6_LMI_RECEIPT_INVALID**, **T7_RISK_ABOVE_THRESH**, **T8_UNKNOWN_BUDGET**.

I13 WSTS (deterministic exploration):
Checklist **W1–W8** in Subsec. 4.4. Key items: WQO certificate; *both* transition and encoder monotonicity; soundness of acceleration (finite-basis/widening); canonical BFS/DFS with `lex_bytes`; upward-closed goal via minimal antichain; visited-sequence hash; pricing by `nodes_upper`. Failure codes: **W1_WQO_INVALID**, **W2_MONOTONICITY_FAIL**, **W3_ACCEL_UNSOUND**, **W4_GOAL_SCHEMA_INVALID**, **W5_EXPLORATION_LIMIT**, **W6_COMMIT_MISMATCH**, **W7_UNKNOWN_BUDGET**.

Loader coupling (non-normative summary)

Given **TC1–TC10**, the Loader treats family verdicts as pure functions of the bound bytes and `EnvLock` identity. Progress is measured by the exposed ρ with No-Zeno fallback; acceptance (and

TailReceipt emission) may decrease audit rank components by binding new evidence, while never trading against higher-priority invariants. All pricing and axis adapters are replayable and versioned via their digests.

Appendix: JSON Schemas (canonical; with shared $defs$)

The following schemas are normative. They use JSON Schema 2020-12 and a shared set of local definitions (`$defs`) for hashes, commits, axis bindings, and normalized work bounds. All objects are canonicalized under `canon/v1`; any additional property is rejected.

Shared $defs

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://specchain.org/schemas/shared-defs.json",
  "$defs": {
    "blake_hash": {
      "type": "string",
      "pattern": "^blake3-256:[A-Za-z0-9+/=._-]{10,}$"
    },
    "axis_binding": {
      "type": "object",
      "required": ["axis_id", "budget_id"],
      "properties": {
        "axis_id":  { "type": "string", "minLength": 1 },
        "budget_id":{ "type": "string", "minLength": 1 }
      },
      "additionalProperties": false
    },
    "expected_work_bound": {
      "type": "object",
      "oneOf": [
        { "required": ["iters_upper"], "properties": { "iters_upper": { "
            type": "integer", "minimum": 1 }}, "additionalProperties":
            false },
        { "required": ["steps_upper"], "properties": { "steps_upper": { "
            type": "integer", "minimum": 1 }}, "additionalProperties":
            false },
        { "required": ["nodes_upper"], "properties": { "nodes_upper": { "
            type": "integer", "minimum": 1 }}, "additionalProperties":
            false }
      ]
    },
    "beta_unknowns_cap": {
      "type": "object",
      "required": ["bytes"],
      "properties": { "bytes": { "type": "integer", "minimum": 0 } },
      "additionalProperties": false
    },
    "solver_commit": {
      "type": "object",
      "required": ["code_hash"],
```

```
      "properties": { "code_hash": { "$ref": "#/$defs/blake_hash" } },
      "additionalProperties": false
    },
    "policy_commit": {
      "type": "object",
      "required": ["code_hash"],
      "properties": { "code_hash": { "$ref": "#/$defs/blake_hash" } },
      "additionalProperties": false
    },
    "operator_commit_raw": {
      "type": "object",
      "required": ["kind","code_hash","param_hash"],
      "properties": {
        "kind":       { "const": "raw" },
        "code_hash":  { "$ref": "#/$defs/blake_hash" },
        "param_hash": { "$ref": "#/$defs/blake_hash" }
      },
      "additionalProperties": false
    },
    "operator_commit_gd": {
      "type": "object",
      "required": ["kind","eta","mu","L","mu_proof_hash","L_proof_hash"],
      "properties": {
        "kind": { "const": "gradient_descent" },
        "eta":  { "type": "number", "exclusiveMinimum": 0 },
        "mu":   { "type": "number", "exclusiveMinimum": 0 },
        "L":    { "type": "number", "exclusiveMinimum": 0 },
        "mu_proof_hash": { "$ref": "#/$defs/blake_hash" },
        "L_proof_hash":  { "$ref": "#/$defs/blake_hash" }
      },
      "additionalProperties": false
    },
    "exploration_strategy": {
      "type": "object",
      "required": ["kind","tie_break"],
      "properties": {
        "kind": { "type": "string", "enum": ["bfs","dfs"] },
        "tie_break": { "type": "string", "enum": ["lex_bytes"] }
      },
      "additionalProperties": false
    }
  }
}
```

ContractionTail.json (FP-safe)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ContractionTail (FP-safe)",
  "type": "object",
  "required": [
    "type","norm_id","center_x_star_digest","ball_radius",
```

```
    "gamma_fp","lipschitz_cert_digest","invariance_cert_digest","
       start_inside_cert_digest",
    "residual_bound","operator_commit","init_state_digest","max_iters",
    "axis_bindings","expected_work_bound","beta_unknowns_cap"
  ],
  "properties": {
    "type": { "const": "ContractionTail" },
    "norm_id": { "type": "string", "enum": ["l2","linf","l1"] },
    "center_x_star_digest": { "$ref": "shared-defs.json#/$defs/blake_hash"
        },
    "ball_radius": { "type": "number", "exclusiveMinimum": 0 },

    "gamma_fp": { "type": "number", "exclusiveMinimum": 0, "
       exclusiveMaximum": 1 },
    "lipschitz_cert_digest":   { "$ref": "shared-defs.json#/$defs/
       blake_hash" },
    "invariance_cert_digest":  { "$ref": "shared-defs.json#/$defs/
       blake_hash" },
    "start_inside_cert_digest":{ "$ref": "shared-defs.json#/$defs/
       blake_hash" },

    "residual_bound": { "type": "number", "exclusiveMinimum": 0 },

    "operator_commit": {
      "oneOf": [
        { "$ref": "shared-defs.json#/$defs/operator_commit_raw" },
        { "$ref": "shared-defs.json#/$defs/operator_commit_gd"  }
      ]
    },

    "init_state_digest": { "$ref": "shared-defs.json#/$defs/blake_hash" },
    "max_iters": { "type": "integer", "minimum": 1 },

    "axis_bindings": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "shared-defs.json#/$defs/axis_binding" }
    },

    "expected_work_bound": { "$ref": "shared-defs.json#/$defs/
       expected_work_bound" },
    "beta_unknowns_cap":   { "$ref": "shared-defs.json#/$defs/
       beta_unknowns_cap" }
  },
  "additionalProperties": false
}
```

ISSTail.json (Q'/uncertainty aware, deterministic & probabilistic)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ISSTail",
  "type": "object",
```

```
"required": [
  "type","clf_cbf_structure","uncertainty_model","tube",
  "policy_commit","dwell_min_steps","accept_criterion",
  "axis_bindings","expected_work_bound","beta_unknowns_cap"
],
"properties": {
  "type": { "const": "ISSTail" },

  "clf_cbf_structure": {
    "type": "object",
    "required": ["V_code_hash","alpha_lower","alpha_upper","rho_decay","
       sigma_forms"],
    "properties": {
      "V_code_hash": { "$ref": "shared-defs.json#/$defs/blake_hash" },
      "alpha_lower": { "type": "number", "exclusiveMinimum": 0 },
      "alpha_upper": { "type": "number", "exclusiveMinimum": 0 },
      "rho_decay":   { "type": "number", "exclusiveMinimum": 0 },
      "sigma_forms": {
        "type": "array", "minItems": 1,
        "items": { "type": "string", "enum": ["affine_disturbance","
           affine_obs","other"] }
      }
    },
    "additionalProperties": false
  },

  "uncertainty_model": {
    "oneOf": [
      {
        "title": "Deterministic intervals",
        "type": "object",
        "required": ["disturbance","observation","Qprime_stance"],
        "properties": {
          "disturbance": { "type": "object",
            "required": ["kind","norm","max"],
            "properties": {
              "kind": { "const": "deterministic" },
              "norm": { "type": "string", "enum": ["l2","linf","l1"] },
              "max":  { "type": "number", "minimum": 0 }
            }, "additionalProperties": false
          },
          "observation": { "type": "object",
            "required": ["norm","max"],
            "properties": {
              "norm": { "type": "string", "enum": ["l2","linf","l1"] },
              "max":  { "type": "number", "minimum": 0 }
            }, "additionalProperties": false
          },
          "Qprime_stance": {
            "type": "object",
            "required": ["carrier","semantics"],
            "properties": {
              "carrier":   { "const": "intervals" },
              "semantics": { "const": "robust" }
```

```
          }, "additionalProperties": false
        }
      },
      "additionalProperties": false
    },
    {
      "title": "Sub-Gaussian disturbance (probabilistic)",
      "type": "object",
      "required": ["disturbance","observation","Qprime_stance"],
      "properties": {
        "disturbance": { "type": "object",
          "required": ["kind","psi2"],
          "properties": {
            "kind": { "const": "subgaussian" },
            "psi2": { "type": "number", "exclusiveMinimum": 0 }
          }, "additionalProperties": false
        },
        "observation": {
          "oneOf": [
            { "type": "object",
              "required": ["kind","norm","max"],
              "properties": {
                "kind": { "const": "bounded" },
                "norm": { "type": "string", "enum": ["l2","linf","l1"]
                    },
                "max":  { "type": "number", "minimum": 0 }
              }, "additionalProperties": false
            },
            { "type": "object",
              "required": ["norm","max"],
              "properties": {
                "norm": { "type": "string", "enum": ["l2","linf","l1"]
                    },
                "max":  { "type": "number", "minimum": 0 }
              }, "additionalProperties": false
            }
          ]
        },
        "Qprime_stance": {
          "type": "object",
          "required": ["carrier","semantics","delta_max"],
          "properties": {
            "carrier":   { "const": "subgaussian" },
            "semantics": { "const": "high-prob" },
            "delta_max": { "type": "number", "exclusiveMinimum": 0, "
               exclusiveMaximum": 1 }
          }, "additionalProperties": false
        }
      },
      "additionalProperties": false
    }
  ]
},
```

```
    "tube": {
      "type": "object",
      "required": ["shape"],
      "properties": {
        "shape":  { "type": "string", "enum": ["polytope","ellipsoid","
            other"] },
        "A_hash": { "$ref": "shared-defs.json#/$defs/blake_hash" },
        "b_hash": { "$ref": "shared-defs.json#/$defs/blake_hash" }
      },
      "additionalProperties": false
    },

    "policy_commit": { "$ref": "shared-defs.json#/$defs/policy_commit" },

    "dwell_min_steps": { "type": "integer", "minimum": 0 },
    "accept_criterion": {
      "type": "object",
      "required": ["in_tube_steps"],
      "properties": { "in_tube_steps": { "type": "integer", "minimum": 1 }
          },
      "additionalProperties": false
    },

    "lmi_witness": {
      "type": "object",
      "required": ["enabled"],
      "properties": {
        "enabled": { "type": "boolean" },
        "problem_digest": { "$ref": "shared-defs.json#/$defs/blake_hash" }
            ,
        "solver_commit":  { "$ref": "shared-defs.json#/$defs/solver_commit
            " },
        "rounding_receipt_hash": { "$ref": "shared-defs.json#/$defs/
            blake_hash" }
      },
      "additionalProperties": false
    },

    "axis_bindings": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "shared-defs.json#/$defs/axis_binding" }
    },

    "expected_work_bound": { "$ref": "shared-defs.json#/$defs/
        expected_work_bound" },
    "beta_unknowns_cap":   { "$ref": "shared-defs.json#/$defs/
        beta_unknowns_cap" }
  },
  "additionalProperties": false
}
```

WSTSTail.json (deterministic exploration + certified acceleration)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "WSTSTail",
  "type": "object",
  "required": [
    "type","state_commit","order_proof","monotonicity_proof",
    "exploration_strategy","acceleration",
    "initial_state_digest","cover_goal","max_explored",
    "axis_bindings","expected_work_bound","beta_unknowns_cap"
  ],
  "properties": {
    "type": { "const": "WSTSTail" },

    "state_commit": {
      "type": "object",
      "required": ["encoder_hash"],
      "properties": { "encoder_hash": { "$ref": "shared-defs.json#/$defs/
         blake_hash" } },
      "additionalProperties": false
    },

    "order_proof": {
      "type": "object",
      "required": ["wqo_certificate_hash"],
      "properties": { "wqo_certificate_hash": { "$ref": "shared-defs.json
         #/$defs/blake_hash" } },
      "additionalProperties": false
    },

    "monotonicity_proof": {
      "type": "object",
      "required": ["transition_checker_hash","encoder_monotonicity_hash"],
      "properties": {
        "transition_checker_hash":   { "$ref": "shared-defs.json#/$defs/
           blake_hash" },
        "encoder_monotonicity_hash": { "$ref": "shared-defs.json#/$defs/
           blake_hash" }
      },
      "additionalProperties": false
    },

    "exploration_strategy": { "$ref": "shared-defs.json#/$defs/
       exploration_strategy" },

    "acceleration": {
      "type": "object",
      "required": ["kind","soundness_cert_hash"],
      "properties": {
        "kind": { "type": "string", "enum": ["finite-basis","widening","
           other"] },
        "soundness_cert_hash": { "$ref": "shared-defs.json#/$defs/
           blake_hash" }
```

```
      },
      "additionalProperties": false
    },

    "initial_state_digest": { "$ref": "shared-defs.json#/$defs/blake_hash"
        },

    "cover_goal": {
      "type": "object",
      "required": ["representation","minimals_hash","minimals_count"],
      "properties": {
        "representation": { "const": "antichain_minimals" },
        "minimals_hash":  { "$ref": "shared-defs.json#/$defs/blake_hash" }
           ,
        "minimals_count": { "type": "integer", "minimum": 1 }
      },
      "additionalProperties": false
    },

    "max_explored": { "type": "integer", "minimum": 1 },

    "axis_bindings": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "shared-defs.json#/$defs/axis_binding" }
    },

    "expected_work_bound": { "$ref": "shared-defs.json#/$defs/
       expected_work_bound" },
    "beta_unknowns_cap":   { "$ref": "shared-defs.json#/$defs/
       beta_unknowns_cap" }
  },
  "additionalProperties": false
}
```

TailReceipt.json (uniform, family-tagged)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "TailReceipt",
  "type": "object",
  "required": [
    "family_id","witness_digest","envlock_id","verdict",
    "work_counters","debit_receipt","axis_debits","unknowns_bytes","
       certs_digests"
  ],
  "properties": {
    "family_id": { "type": "string", "enum": ["I11_Contraction","
       I12_ISS_Tubes","I13_WSTS"] },
    "witness_digest": { "$ref": "shared-defs.json#/$defs/blake_hash" },
    "envlock_id": { "type": "string", "minLength": 1 },

    "verdict": { "type": "string", "enum": ["ACCEPT","REJECT"] },
```

```
"failure_code": { "type": ["string","null"] },

"work_counters": {
  "type": "object",
  "oneOf": [
    { "required": ["iters_realized"], "properties": { "iters_realized"
       : { "type": "integer", "minimum": 0 }}, "additionalProperties":
        false },
    { "required": ["steps_realized"], "properties": { "steps_realized"
       : { "type": "integer", "minimum": 0 }}, "additionalProperties":
        false },
    { "required": ["nodes_explored"], "properties": { "nodes_explored"
       : { "type": "integer", "minimum": 0 }}, "additionalProperties":
        false }
  ]
},

"debit_receipt": {
  "type": "object",
  "required": ["tariff_digest","expected_work_bound","priced_delta_fr"
     ],
  "properties": {
    "tariff_digest": { "$ref": "shared-defs.json#/$defs/blake_hash" },
    "expected_work_bound": { "$ref": "shared-defs.json#/$defs/
       expected_work_bound" },
    "priced_delta_fr": { "type": "number", "minimum": 0 }
  },
  "additionalProperties": false
},

"axis_debits": {
  "type": "object",
  "additionalProperties": { "type": "number", "minimum": 0 }
},

"unknowns_bytes": { "type": "integer", "minimum": 0 },

"certs_digests": {
  "type": "array",
  "items": { "$ref": "shared-defs.json#/$defs/blake_hash" }
},

"exploration_profile": {
  "type": ["object","null"],
  "properties": {
    "visited_sequence_hash": { "$ref": "shared-defs.json#/$defs/
       blake_hash" },
    "dwell_receipt": {
      "type": "object",
      "required": ["local_dwell","authorized_reset"],
      "properties": {
        "local_dwell": { "type": "integer", "minimum": 0 },
        "authorized_reset": { "type": "boolean" }
      },
```

```
          "additionalProperties": false
        }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```